Thesis outline and source pack

Latency and Accuracy Optimization for OCR to LLM Document Processing Pipelines – chapter plan in the order the committee asked for, with the facts, figures and sources each chapter can draw on

How to use this. Each chapter lists what it must answer, which measured facts and figures support it, what to read, and what the committee will check. Every number here comes from `docs/Architecture_and_Benchmark_Report.pdf` and the output folders named in it; re-check numbers against those folders before submission. `docs/Literature_Sources.pdf` holds the verified sources and `docs/Requirements_Compliance.pdf` maps every committee and supervisor requirement to the code. Read every source yourself before citing it.

#	Chapter	IACS component	Committee feedback it answers
0	Introduction	–	State the problem, goal and tasks; do not claim to be the first
1	Analytical part: problem, existing solutions, technology choices	A-3, A-4	"Complete lack of an analytical section", "very few references", VLMs ignored
2	Solution: system design of Architectures A and B	S-1	Formal routing algorithm; specific tools (bitsandbytes), not generic words
3	Dataset and evaluation methodology	Verification	"Dataset introduced after the results" – must come before results
4	Experimental results	Verification	"Some results are incorrect due to a bug" – re-run on the scaled dataset
5	Discussion and conclusions	–	Trade-offs and future work, tied to measurements

0. Introduction

Sets up the problem and the thesis goal in one to two pages.

Must answer

- Why invoice data extraction matters (manual entry cost, volume) and why OCR → LLM pipelines are used for it.
- What is slow or expensive about such pipelines on modest hardware (latency, throughput, memory, cost).
- The goal and the six tasks exactly as approved by the faculty; the thesis type (Type 2 – solution to a current problem).
- The contribution in one paragraph and the structure of the remaining chapters.

Facts and numbers to use

- Goal: design and compare a sequential and an optimised asynchronous OCR → LLM architecture, improving latency, throughput and cost while keeping accuracy.
- Headline result to state in the abstract: B processes 301.7 invoices/min vs 53.1 for A (5.7×) at the same F1 (0.941), on an RTX 3050 laptop GPU with 4 GB; cost per 100 invoices €0.0019 vs €0.0110.

The committee will check

- No claim to be the first to optimise such pipelines – position the work relative to chapter 1.

1. Analytical part (A-3, A-4)

The section the committee missed most. Analyse the problem, review other authors' solutions and justify every technology choice against alternatives.

Must answer

- Task 1: what limits existing OCR → LLM pipelines – latency, cost, scalability – with evidence from the literature.
- Task 2: review of OCR tools, caching methods, compression techniques and asynchronous processing approaches.
- Why a dedicated OCR → LLM pipeline is still justified when vision-language models (VLMs) read documents directly – the committee's explicit question.
- A comparison table per technology decision: chosen tool, alternatives, criteria, reason (supervisor at 20 %: e.g. Redis vs Memcached, Celery vs Kafka).
- The research gap that motivates the thesis.

Facts and numbers to use

- OCR: Tesseract 5 chosen; it runs on the CPU and leaves the 4 GB GPU to the language model, and its word-level confidence feeds the routing and review rules.
- OCR engines, measured on all 600 invoices with the same downstream and scorer: Tesseract F1 0.941, 0.89 s per real invoice on one CPU thread, no GPU memory; EasyOCR F1 0.934, 8.0 s per real invoice on the GPU with a 3.8 GB peak – nearly the whole card (outputs/2026-10-20_01-52-39_ocr_comparison).
- Pipeline memory measured on the thesis GPU: 1.07 GB (A) and 2.0 GB (B) peak.
- VLM measured on the thesis GPU: Qwen2-VL-2B in 4-bit NF4 (16-bit weights do not fit), at most 1024 visual tokens, reads the image directly: F1 0.966 (real 0.930), 3.9 s per invoice, GPU peak 1745 MB – against B: F1 0.941, 0.20 s of wall time per invoice, 1996 MB (outputs/2026-10-20_23-54-32_vlm_baseline).
- Resolution probe (20 invoices, one setting per process): ≤ 512 tokens: F1 0.95, 3.2 s, 1568 MB; ≤ 1024 tokens: F1 1.00, 3.8 s, 1745 MB; ≤ 2048 tokens: F1 0.95, 5.2 s, 2329 MB (outputs/2026-10-21_00-35-48_vlm_resolution_probe_20docs).
- The VLM is more accurate here; the argument for OCR on this hardware is throughput, latency and cost – see Architecture_and_Benchmark_Report.pdf section 7 – and the literature list has the counter-evidence the thesis must answer.
- Language model: Flan-T5-base (248 M parameters) fits in fp32, so A vs B compares architecture, not memory starvation; its 512-token limit is why the embedding filter exists.
- Caching: SQLite chosen over Redis/Memcached – one workstation, no server process, durable, lookup ≈ 1 ms per invoice (0.59 s for all 600). Redis becomes the better choice with several machines.
- Queue/concurrency: Python asyncio + process pool + one GPU thread chosen over Celery/Kafka – single node; a broker adds operations cost without throughput gain here.
- Compression: bitsandbytes LLM.int8 and NF4 (4-bit, double quantisation) evaluated; GGUF/llama.cpp is the alternative for CPU or larger decoder models.
- Commercial invoice parsers (AWS, Azure, Google) keep text layout and validate totals arithmetically – the same two techniques that raised F1 from 0.85 to 0.94 here.

Figures and tables available

- Architecture overview diagrams (Figures 1–2 of the Architecture Report) can preview the solution at the end of the chapter.

Read before writing

- **docs/Literature_Sources.pdf**: about 150 verified sources with summaries – comparable bachelor's and master's theses (section 1), VLMs vs OCR pipelines including counter-evidence (section 2), OCR engines (3), LLM-based invoice extraction and metrics (4), batching, caching, quantization and concurrency (5), queueing theory (6). Start with the □ sources.
- OCR: Smith (2007) An Overview of the Tesseract OCR Engine, ICDAR. EasyOCR and PaddleOCR documentation for the alternatives.
- Layout-aware models: Xu et al. (2020) LayoutLM, KDD; Huang et al. (2022) LayoutLMv3, ACM Multimedia; Wang et al. (2023) DocLLM, arXiv 2401.00908; Perot et al. (2023) LMDX, arXiv 2309.10952.
- OCR-free models and VLMs: Kim et al. (2022) Donut, ECCV; Lee et al. (2023) Pix2Struct, ICML; Wang et al. (2024) Qwen2-VL, arXiv 2409.12191.
- Language models: Raffel et al. (2020) T5, JMLR; Chung et al. (2022) Scaling Instruction-Finetuned Language Models (Flan-T5), arXiv 2210.11416.
- Embeddings: Reimers and Gurevych (2019) Sentence-BERT, EMNLP; Wang et al. (2020) MiniLM, NeurIPS.
- Compression: Dettmers et al. (2022) LLM.int8(), NeurIPS; Dettmers et al. (2023) QLoRA, NeurIPS; Frantar et al. (2023) GPTQ, ICLR.
- Batching and serving: Yu et al. (2022) Orca, OSDI; Kwon et al. (2023) vLLM / PagedAttention, SOSP.
- Datasets for related work: Park et al. (2019) CORD; Jaume et al. (2019) FUNSD; Huang et al. (2019) ICDAR 2019 SROIE competition.
- Industry: AWS Textract AnalyzeExpense, Azure AI Document Intelligence prebuilt invoice model, Google Document AI Invoice Parser – product documentation.

The committee will check

- Enough references from peer-reviewed work, not only blogs.
- That VLMs are discussed and the decision for OCR → LLM is argued with hardware, cost and control – ideally with numbers.
- That each tool has been compared with alternatives, not just named.

2. Solution: system design (S-1)

Describe both architectures precisely enough to rebuild them. Source: docs/ARCHITECTURE.md and the code in pipeline/.

Must answer

- Requirements derived from chapter 1.
- Architecture A and Architecture B: components, data flow, what runs where (process, thread, GPU).
- The algorithms, each as pseudocode with its parameters: row-based text reconstruction, embedding line selection, routing rule, total reconciliation, review flag, cache key.
- How quantisation is configured (bitsandbytes settings) and why 4-bit NF4 and 8-bit were both evaluated.

Facts and numbers to use

- A: OCR → MiniLM line filter → Flan-T5 (3 sequential calls) → total check, one invoice at a time, fp32.
- B: SHA-256 cache lookup → 3 OCR worker processes → asyncio forms batches of 8 → one GPU thread: batched filter, router, 4-bit (FAST, 12-token answers) or fp32 (FULL, 24-token answers) → total check → cache write.
- Routing rule: FAST if token_count ≤ T and OCR confidence ≥ C, else FULL; T and C derived from data (current code: T = 525, C = 70).
- Row reconstruction: group Tesseract line segments whose vertical centres differ by less than 0.6 × median line height; join left to right with " | ".
- Reconciliation: on lines containing total / amount due / gross, if amounts a + b = c (±0.01), the total is c; otherwise keep the model's answer.
- Review flag: missing or unreadable field, total not found on a total line, or mean OCR confidence < 80.
- Quantisation: BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=float16); 8-bit with load_in_8bit=True.

Figures and tables available

- Figures 1 and 2 (architecture diagrams) and the side-by-side table in the Architecture Report, section 2.
- The defects-fixed table (Architecture Report 3.1) belongs in an appendix or a short "previous implementation" note, not here.

The committee will check

- That the text matches the code: SQLite (not Redis), asyncio + process pool (not "pure asyncio"), question-answer prompts (not JSON), thresholds derived (not guessed).
- That the routing logic is a formal algorithm with mathematical thresholds – asked for at 20 %, 25 % and 75 %.

3. Dataset and evaluation methodology

Must come before the results. Source: docs/EVALUATION.md.

Must answer

- Where the 600 invoices come from, how ground truth was created and corrected.
- How accuracy is measured (F1 with fuzzy matching) and why; how speed, memory and cost are measured.
- The experiment design: configurations, repetitions, hardware and software, controls against measurement noise.

Facts and numbers to use

- 600 invoices: 300 real (1–15 and 316–600, hand-labelled), 300 synthetic (16–315, generated by generate_dataset.py).
- Ground truth: total means the gross amount due; 26 labels that held the net total were corrected, each verified by net + VAT = gross; originals kept.
- Scoring: per field TP / FP / FN / TN; a wrong value counts as FP and FN; precision, recall, F1 micro-averaged; invoice numbers fuzzy-matched at RapidFuzz ratio ≥ 90, dates compared as calendar dates, amounts as numbers.
- Throughput = invoices ÷ wall-clock time; service time vs latency; pipeline GPU memory (PyTorch peak) and RAM of all processes, sampled 4×/s; cost = wall time × €0.35 per GPU-hour.
- 11 configurations (A, B and nine one-change variants of B), 3 runs each, mean ± standard deviation.
- Hardware: RTX 3050 Laptop GPU 4 GB, 12 logical CPU cores, 16 GB RAM, Windows 11; Python 3.12, PyTorch 2.5.1 + CUDA 12.1, transformers 5.7, bitsandbytes 0.50, Tesseract 5.4.
- Measurement controls: laptop plugged in and kept awake, offline model loading, no other workloads; runs disturbed by other programs were detected by OCR time per invoice and repeated (kept in a discarded/ folder for transparency).
- Three further experiments, each with its own method in docs/EVALUATION.md section 7: latency with invoices arriving one at a time (Poisson arrivals, 4 rates, 3 runs), Tesseract vs EasyOCR (all 600 invoices, same downstream), and a VLM baseline (same invoices and scorer).

Figures and tables available

- Configuration table (run_info.json and summary.md of the benchmark folder).
- Hardware and software table (run_info.json).

The committee will check

- That the ground-truth method is explained, including the corrections.
- That the order is method → results, not results → dataset.

4. Experimental results (solution verification)

Present the measurements, then interpret them. Source: summary.md, figures/ and routing_analysis.md in the final benchmark folder, and the Architecture Report section 5.

Must answer

- A vs B on speed, memory and cost; where B's gain comes from (ablation).
- Accuracy, overall and per field, real vs synthetic; the effect of the extraction upgrade with the error analysis that motivated it.
- The quantisation trade-off and the routing-threshold study.
- Human review: share processed without a person, accuracy of what is accepted.
- Threats to validity.

Facts and numbers to use

- A: 677.9 ± 7.4 s for 600 invoices (53.1/min); B: 119.4 ± 4.7 s (301.7/min) – 5.7×; warm cache 0.59 s for all 600 (3 runs each).
- Per invoice: OCR 0.59 s (A) and 0.58 s (B); LLM 0.51 s vs 0.13 s. Cost per 100 invoices €0.0110 vs €0.0019. Pipeline GPU memory 1067 vs 1996 MB.
- F1 0.941 for both A and B (real 0.89, synthetic 0.99); before the extraction upgrade 0.847 / 0.850.
- Error analysis that justified the upgrade: in 154 of 199 wrong totals the right amount was already in the model's input; the model picked the net. Row layout + reconciliation fixed 146 (121 on labels never changed).
- Human review: 92.5 % accepted automatically, 90.5 % of those fully correct (84.2 % without the flag); 93 % of flagged invoices really were wrong.
- Routing: the data-driven thresholds send 99.8 % of invoices to the 4-bit model; 4-bit and fp32 disagree on only 10 invoices (5 each way).
- Ablation: 1 OCR worker 338.4 s (parallel OCR worth 2.8×); no batching 248.8 s (batching worth 2.1×); 6 workers 102.9 ± 10.3 s (16 % faster, less stable, more RAM); no cache 126.0 s (no cost on a first pass).
- Precision (every invoice on one model): fp32 126.1 s / 1645 MB; fp16 123.5 s / 1049 MB; 4-bit 125.8 s / 1102 MB; 8-bit 167.8 s / 1176 MB; F1 0.941 except 8-bit (0.939, varies slightly between runs).
- Measurement incidents to report: runs overlapped by other programs (two demo runs, a runaway background process) were detected through OCR time per invoice and repeated; the 33 discarded runs are kept for transparency.
- Latency with invoices arriving one at a time (60 invoices, Poisson arrivals at 30, 48, 120, 240 per minute, 3 runs): A median 1.7 s at 30/min but 30 s at 240/min (queue grows above ~53/min); B as benchmarked 7.9 s at 30/min (waits for 8 invoices); B with dynamic batching 0.95–1.8 s at every rate; F1 identical (outputs/2026-10-20_01-01-33_latency_60docs).
- OCR engines, measured on all 600 invoices with the same downstream and scorer: Tesseract F1 0.941, 0.89 s per real invoice on one CPU thread, no GPU memory; EasyOCR F1 0.934, 8.0 s per real invoice on the GPU with a 3.8 GB peak – nearly the whole card (outputs/2026-10-20_01-52-39_ocr_comparison).
- VLM measured on the thesis GPU: Qwen2-VL-2B in 4-bit NF4 (16-bit weights do not fit), at most 1024 visual tokens, reads the image directly: F1 0.966 (real 0.930), 3.9 s per invoice, GPU peak 1745 MB – against B: F1 0.941, 0.20 s of wall time per invoice, 1996 MB (outputs/2026-10-20_23-54-32_vlm_baseline).
- Resolution probe (20 invoices, one setting per process): ≤ 512 tokens: F1 0.95, 3.2 s, 1568 MB; ≤ 1024 tokens: F1 1.00, 3.8 s, 1745 MB; ≤ 2048 tokens: F1 0.95, 5.2 s, 2329 MB (outputs/2026-10-21_00-35-48_vlm_resolution_probe_20docs).
- The VLM is more accurate here; the argument for OCR on this hardware is throughput, latency and cost – see Architecture_and_Benchmark_Report.pdf section 7 – and the literature list has the counter-evidence the thesis must answer.
- Scoring correction found during the OCR comparison: dates written without a space after the comma ('April 19,2024') were parsed with the current year; fixed, unit-tested, all runs re-scored; the main benchmark did not change.

Figures and tables available

- throughput.png, wall_time.png, service_time.png, stage_breakdown.png, memory.png, cost.png, f1.png, f1_by_source.png, precision_tradeoff.png, routing_scatter.png (benchmark folder, figures/).
- All tables in summary.md; routing grid in routing_analysis.md.

The committee will check

- That every number matches the benchmark output – this is where the earlier bug was caught.
- That accuracy is reported for real and synthetic invoices separately.
- That disturbed runs, label corrections and other incidents are reported, not hidden.

5. Discussion and conclusions

Answer the research question and discuss the trade-offs honestly.

Must answer

- Did the optimised architecture improve latency, throughput and cost while keeping accuracy? With numbers.
- Which optimisations mattered and which did not – and why (OCR parallelism and batching vs quantisation and caching on a first pass).
- Throughput vs latency in a batched system; memory vs speed in quantisation.
- Limitations and future work.

Facts and numbers to use

- Quantisation saved GPU memory but did not speed up a 248 M-parameter model on this GPU; fp16 was at least as fast and used the least memory.
- The cache pays off only for repeated invoices; on a first pass it costs nothing measurable.
- Throughput vs latency: B's batching raises throughput 5.7× but, waiting for 8 invoices, makes light-load latency worse than A's; dynamic batching keeps the throughput and the low latency (chapter 4 latency facts).
- Label checks (invoice number and date must sit on a labelled line), estimated from stored results: wrong invoices caught 42 → 64 of 95; fully correct among auto-accepted 90.5 % → 94.0 %; straight-through 92.5 % → 86.2 %.
- OCR pipeline plus VLM for flagged invoices only, estimated: fully correct 505 → 541 of 600 for about 0.29 s of extra GPU time per invoice.
- Limitations: one small language model; synthetic half is easy; single laptop; one VLM with one prompt; the latency study uses 60 invoices.
- Future work: dynamic batching as B's default; a faster OCR stage (B's bottleneck); label checks for invoice number and date; routing between genuinely different models (Flan-T5 and a VLM for flagged invoices); distributed cache and queue for several machines.

The committee will check

- That conclusions follow from the measured results and do not overclaim.

Appendices worth including

- How to reproduce: the commands from the README (unit tests, demo, benchmark), and where results are written.
- The previous implementation and its defects (Architecture Report 3.1) – shows how the earlier incorrect results arose and were fixed.
- Corrected ground-truth labels (list of the 26 invoices).
- Full result tables per configuration (summary.md).